

MySQL Employee Audit & Change Tracking System

Project Documentation — SRE-Oriented Database Auditing & Troubleshooting

1. Project Overview

The MySQL Employee Audit & Change Tracking System automatically records important changes made to employee records. MySQL triggers create audit entries whenever records are inserted, updated, or deleted. The audit trail is intended to support practical investigation: identifying what changed, what the previous value was, what the new value is, when it changed, and which database account performed it.

2. Problem Statement

When employee data changes, the main table stores the current state but does not automatically preserve every previous value. This can make unexpected changes difficult to investigate. A separate audit log preserves selected historical values and creates a trace of database activity.

3. Objectives

- Import an employee dataset into MySQL and store it in a structured table.
- Automatically log INSERT, UPDATE, and DELETE operations with MySQL triggers.
- Capture OLD and NEW values for updates, plus user and timestamp information.
- Use SQL queries to investigate and summarize audit activity.

4. Dataset & Tools

Table 1.1. Dataset and environment used for the implementation.

Item	Details
Dataset	Employee Records Dataset — Kaggle
Records	30,000 synthetic employee records
Main fields	Employee ID, Name, Age, Country, Department, Position, Salary, Joining Date
Database / tool	MySQL 8.0 / MySQL Workbench

Source: [Kaggle — Employee Records Dataset](#). The Kaggle data card describes the dataset as synthetic and containing 30,000 employee records.

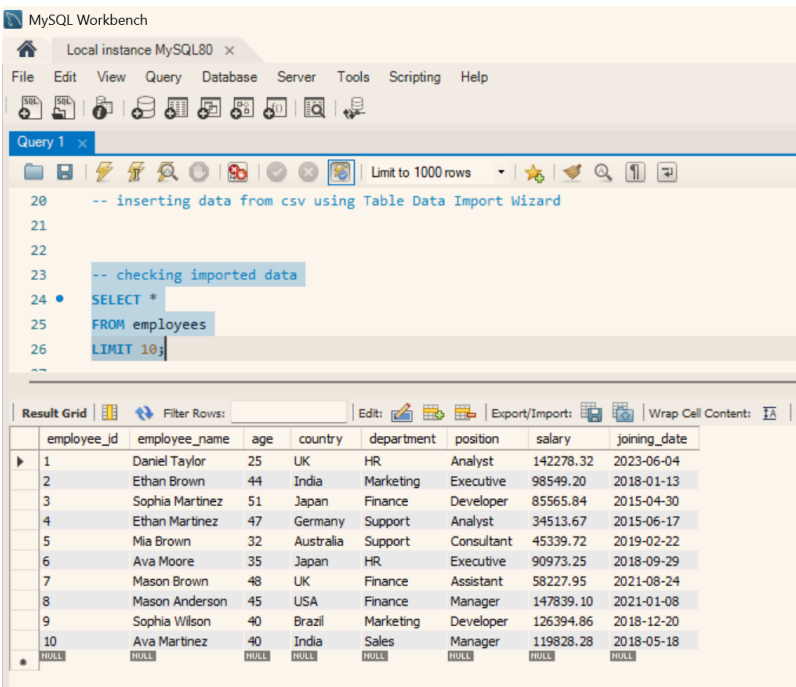


Figure 1.1. MySQL Workbench showing the imported employee dataset and a validation query using SELECT with LIMIT.

5. Database Design

The implementation uses two main tables. The employees table represents the current employee state, while audit_log stores historical information about tracked changes.

5.1 Employees Table

Table 1.2. Structure of the employees table used to store the current employee state.

Column	Type	Purpose
employee_id	INT	Primary key / unique employee ID
employee_name	VARCHAR(100)	Employee name
age	INT	Employee age
country	VARCHAR(50)	Country of employment
department	VARCHAR(100)	Current department
position	VARCHAR(100)	Current job position
salary	DECIMAL(12,2)	Current salary
joining_date	DATE	Employee joining date

5.2 Audit Log Table

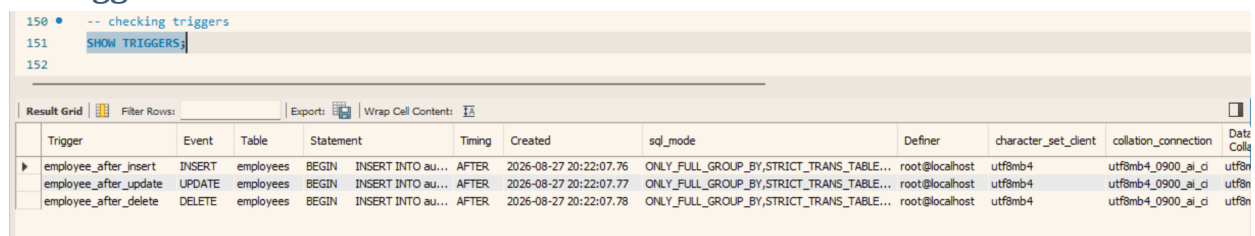
Table 1.3. Fields captured by audit_log for change tracking and investigation.

Column	Purpose
log_id	Unique audit record ID
employee_id	Employee affected by the operation
action_type	INSERT, UPDATE, or DELETE
old_salary / new_salary	Salary before and after an update
old_department / new_department	Department before and after an update
old_position / new_position	Position before and after an update
changed_by	MySQL account that performed the operation
changed_at	Timestamp of the audit event

6. Trigger-Based Audit Flow

- INSERT: a new row is added → the INSERT trigger writes the new values to audit_log.
- UPDATE: a row changes → the UPDATE trigger records OLD and NEW values.
- DELETE: a row is removed → the DELETE trigger records the values that existed before deletion.
- Investigation: SQL queries filter and aggregate the audit history.

6.1 Trigger Verification



150	-- checking triggers
151	SHOW TRIGGERS;
152	

Trigger	Event	Table	Statement	Timing	Created	sql_mode	Definer	character_set_client	collation_connection	Data Coll
employee_after_insert	INSERT	employees	BEGIN INSERT INTO au...	AFTER	2026-08-27 20:22:07.76	ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLE...	root@localhost	utf8mb4	utf8mb4_0900_ai_ci	utf8m
employee_after_update	UPDATE	employees	BEGIN INSERT INTO au...	AFTER	2026-08-27 20:22:07.77	ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLE...	root@localhost	utf8mb4	utf8mb4_0900_ai_ci	utf8m
employee_after_delete	DELETE	employees	BEGIN INSERT INTO au...	AFTER	2026-08-27 20:22:07.78	ONLY_FULL_GROUP_BY,STRICT_TRANS_TABLE...	root@localhost	utf8mb4	utf8mb4_0900_ai_ci	utf8m

Figure 1.2. MySQL Workbench output from SHOW TRIGGERS confirming the INSERT, UPDATE, and DELETE triggers.

7. Trigger Implementation & Testing

7.1 INSERT Trigger

The INSERT trigger runs after a new employee is added. It uses NEW because the inserted row has no previous version.

```
AFTER INSERT ON employees
INSERT INTO audit_log (... , new_salary, new_department, new_position, changed_by)
VALUES (... , NEW.salary, NEW.department, NEW.position, CURRENT_USER())
```

7.2 UPDATE Trigger — OLD vs NEW

The UPDATE trigger is the key audit mechanism. OLD represents the row before the update and NEW represents the row after the update. Recording both allows the change itself to be investigated.

```
old_salary = OLD.salary
new_salary = NEW.salary
old_department = OLD.department
new_department = NEW.department
```

```

154  -- testing update operation
155  • UPDATE employees
156  SET salary = salary + 5000
157  WHERE employee_id = 1;
158
159
160  -- checking audit log after update
161  • SELECT *
162  FROM audit_log
163  ORDER BY changed_at DESC;
164

```

log_id	employee_id	action_type	old_salary	new_salary	old_department	new_department	old_position	new_position	changed_by	changed_at
5	1	UPDATE	147278.32	152278.32	HR	HR	Analyst	Analyst	root@localhost	2026-08-27 20:24:14
1	1	UPDATE	142278.32	147278.32	HR	HR	Analyst	Analyst	root@localhost	2026-08-27 20:22:07
2	2	UPDATE	98549.20	98549.20	Marketing	Engineering	Executive	Executive	root@localhost	2026-08-27 20:22:07
3	3	UPDATE	85565.84	85565.84	Finance	Finance	Developer	Senior Analyst	root@localhost	2026-08-27 20:22:07
4	4	DELETE	34513.67	NULL	Support	NULL	Analyst	NULL	root@localhost	2026-08-27 20:22:07

Figure 1.3. Testing an UPDATE operation and querying audit_log; the result shows OLD and NEW values captured by the trigger.

```

171
172  -- checking audit log
173  • SELECT *
174  FROM audit_log
175  ORDER BY changed_at DESC;
176

```

log_id	employee_id	action_type	old_salary	new_salary	old_department	new_department	old_position	new_position	changed_by	changed_at
6	2	UPDATE	98549.20	98549.20	Engineering	Engineering	Executive	Executive	root@localhost	2026-08-27 20:24:48
5	1	UPDATE	147278.32	152278.32	HR	HR	Analyst	Analyst	root@localhost	2026-08-27 20:24:14
1	1	UPDATE	142278.32	147278.32	HR	HR	Analyst	Analyst	root@localhost	2026-08-27 20:22:07
2	2	UPDATE	98549.20	98549.20	Marketing	Engineering	Executive	Executive	root@localhost	2026-08-27 20:22:07
3	3	UPDATE	85565.84	85565.84	Finance	Finance	Developer	Senior Analyst	root@localhost	2026-08-27 20:22:07
4	4	DELETE	34513.67	NULL	Support	NULL	Analyst	NULL	root@localhost	2026-08-27 20:22:07

Figure 1.4. Audit log after additional update activity; the log preserves operation type, previous values, new values, user, and timestamp.

7.3 DELETE Trigger

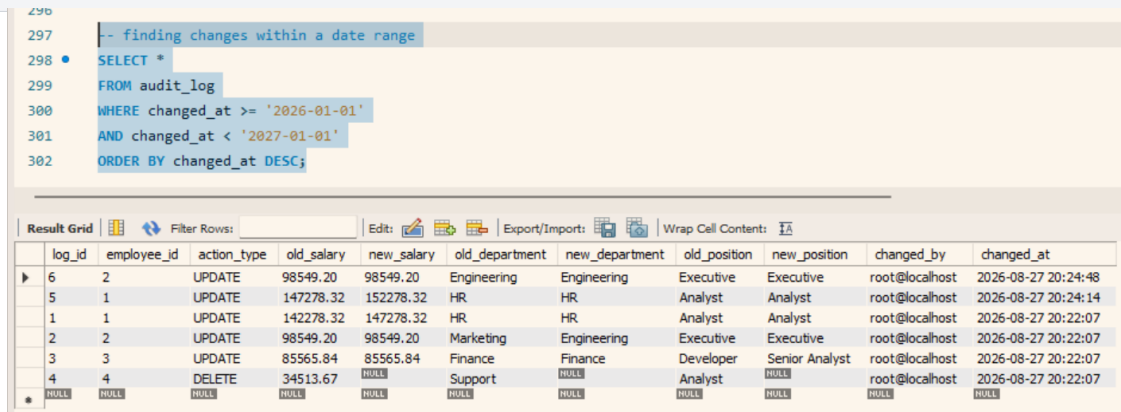
For a DELETE operation there is no NEW row. The trigger therefore uses OLD values to preserve the employee information that existed immediately before deletion.

8. Audit Investigation Queries

The audit table can be treated as a basic investigation source. The project includes queries for salary changes, department and position changes, recent activity, employee-level modification frequency, and date-range filtering.

8.1 Date-Range Investigation

```
SELECT *
FROM audit_log
WHERE changed_at >= '2026-01-01'
AND changed_at < '2027-01-01'
ORDER BY changed_at DESC;
```



The screenshot shows a MySQL Workbench interface. The query editor contains the following SQL code:

```
-- finding changes within a date range
SELECT *
FROM audit_log
WHERE changed_at >= '2026-01-01'
AND changed_at < '2027-01-01'
ORDER BY changed_at DESC;
```

The result grid below the query editor displays the following data:

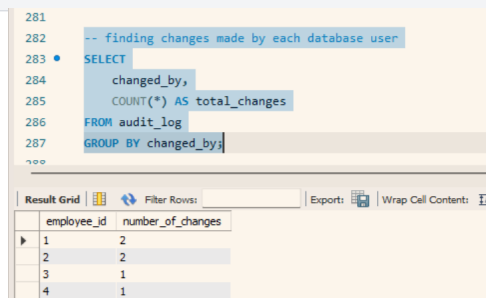
log_id	employee_id	action_type	old_salary	new_salary	old_department	new_department	old_position	new_position	changed_by	changed_at
6	2	UPDATE	98549.20	98549.20	Engineering	Engineering	Executive	Executive	root@localhost	2026-08-27 20:24:48
5	1	UPDATE	147278.32	152278.32	HR	HR	Analyst	Analyst	root@localhost	2026-08-27 20:24:14
1	1	UPDATE	142278.32	147278.32	HR	HR	Analyst	Analyst	root@localhost	2026-08-27 20:22:07
2	2	UPDATE	98549.20	98549.20	Marketing	Engineering	Executive	Executive	root@localhost	2026-08-27 20:22:07
3	3	UPDATE	85565.84	85565.84	Finance	Finance	Developer	Senior Analyst	root@localhost	2026-08-27 20:22:07
4	4	DELETE	34513.67	NULL	Support	NULL	Analyst	NULL	root@localhost	2026-08-27 20:22:07

Figure 1.5. Filtering audit activity within a specified date range and ordering the events by most recent timestamp.

8.2 Activity Analysis

GROUP BY and COUNT can be used to identify employees with frequent modifications, helping prioritize investigation of records with unusually high change activity.

```
SELECT employee_id, COUNT(*) AS number_of_changes
FROM audit_log
GROUP BY employee_id
ORDER BY number_of_changes DESC;
```



The screenshot shows a MySQL Workbench interface. The query editor contains the following SQL code:

```
-- finding changes made by each database user
SELECT
  changed_by,
  COUNT(*) AS total_changes
FROM audit_log
GROUP BY changed_by;
```

The result grid below the query editor displays the following data:

employee_id	number_of_changes
1	2
2	2
3	1
4	1

Figure 1.6. MySQL Workbench showing an audit-activity aggregation query. The visible result grid reflects the employee-level change-count query.

Accuracy note: The screenshot contains a query prepared for grouping by database user, but its visible result grid shows employee-level change counts. It is therefore treated as evidence of audit-query execution, not as evidence of the database-user aggregation result.

8.3 Troubleshooting Scenario

If an employee's salary changes unexpectedly, the audit log can be queried by employee ID and UPDATE action. The investigator can compare OLD and NEW salary, inspect the recorded database user, and use the timestamp to establish when the change occurred.

9. SRE Relevance & Project Outcomes

The project demonstrates an operational approach to database changes: preserve useful evidence, automate logging at the database layer, and use SQL to investigate unexpected activity. This maps naturally to troubleshooting and root-cause investigation workflows.

9.1 Skills Demonstrated

Table 1.4. Technical and operational capabilities demonstrated by the project.

Area	Demonstrated capability
MySQL	Database creation, tables, CRUD operations, triggers, timestamps, user functions
SQL	Filtering, sorting, aggregation, GROUP BY, COUNT, date-range analysis
Auditing	Automatic logging of INSERT, UPDATE, and DELETE operations
Troubleshooting	Using historical changes to investigate unexpected data modifications
Operational thinking	Separating current state from historical audit information

9.2 Key Learnings

- Triggers can automate database-side actions without requiring application code for each change.
- OLD and NEW values make UPDATE operations auditable and reconstructable.
- A separate audit table preserves history while the main table represents current state.
- Audit records can be analyzed with SQL to identify recent changes and activity patterns.

9.3 Limitations & Future Improvements

- The project tracks selected employee fields rather than every possible database change.
- It is a learning project and does not implement enterprise-scale monitoring, alerting, or centralized log management.
- Future extensions could include audit views, dashboards, suspicious-activity alerts, and integration with application/infrastructure logs.

9.4 Resume Description

MySQL Employee Audit & Change Tracking System

- Built a MySQL audit logging system using INSERT, UPDATE and DELETE triggers to automatically track employee data modifications.
- Captured old/new values, database users and timestamps to support change tracking and troubleshooting.
- Developed SQL queries to investigate salary, department and position changes and analyze database activity.

Dataset Link

[Kaggle — Employee Records Dataset](#)

Recommended repository structure: one complete SQL file (for example, `employee_audit_system.sql`) plus `README.md`. The SQL file can contain database creation, table creation, trigger definitions, testing, and analysis queries in execution order.